# UNIT - V : INTRODUCTION TO DATA SCIENCE

## 1. Introduction to data science

* DATA SCIENCE = extracting knowledge and insights from structured and unstructured data using statistics, programming and domain knowledge. Life cycle: collect data -> clean (pre-process) -> explore / analyse -> model -> visualise -> communicate / deploy.
* Python is popular for data science because of simple syntax and powerful libraries: NumPy (arrays), Pandas (tables), Matplotlib / Seaborn (plots), SciPy, scikit-learn (machine learning).

## 2. Functional Programming

* A programming style that builds programs by applying and composing FUNCTIONS, avoiding changing state and mutable data. Features: PURE functions (same input -> same output, no side effects), first class functions (assign to variables, pass as arguments, return from functions), higher order functions, immutability, recursion, lazy iterators.

### Higher order functions - map, filter, reduce

| Function | Action | Result |
|---|---|---|
| map(func, iter) | Apply func to every item | map object (iterator) |
| filter(func, iter) | Keep items for which func is True | filter object |
| reduce(func, iter) | Cumulatively combine to a single value (functools) | single value |

```
from functools import reduce
nums = [1, 2, 3, 4, 5]
sq   = list(map(lambda x: x * x, nums))          # [1,4,9,16,25]
ev   = list(filter(lambda x: x % 2 == 0, nums))  # [2, 4]
tot  = reduce(lambda a, b: a + b, nums)          # 15
prod = reduce(lambda a, b: a * b, nums)          # 120
print(sq, ev, tot, prod)
print(sorted(["bb", "a", "ccc"], key=len))       # ['a','bb','ccc']
```

* Other tools: lambda, list comprehension, generators (yield), iterators, zip, enumerate, itertools, functools.partial, decorators (functions that wrap other functions), closures.

```
def gen(n):                          # generator - lazy sequence
    for i in range(n):
        yield i * i
print(list(gen(5)))                  # [0, 1, 4, 9, 16]

def make_adder(x):                   # closure / function returning functi
    return lambda y: x + y
add5 = make_adder(5); print(add5(3))    # 8
```

# 3. JSON and XML in Python

## JSON (JavaScript Object Notation)

* Lightweight, text based, language independent data-interchange format; looks like a Python dict: { key: value }. Data types: object, array, string, number, true / false, null. Python's json module:

json.dumps(obj) -> JSON string, json.loads(str) -> Python object, json.dump(obj, file), json.load(file).

| JSON | Python |
|---|---|
| object { } | dict |
| array [ ] | list |
| string | str |
| number (int / real) | int / float |
| true / false | True / False |
| null | None |

```
import json
data = {"name": "Anu", "age": 20, "skills": ["C", "Python"], "ok": Tru
s = json.dumps(data, indent=2)       # dict -> JSON string
print(s)
obj = json.loads('{"a": 1, "b": [1, 2], "c": null}')   # string -> dic
print(obj["b"], obj["c"])            # [1, 2] None
with open("d.json", "w") as f:
    json.dump(data, f)               # write to file
with open("d.json") as f:
    back = json.load(f)              # read from file
```

## XML (eXtensible Markup Language)

* Markup language that stores data in a tree of user-defined TAGS with attributes: <student id='1'><name>Anu</name></student>. Python module: xml.etree.ElementTree - ET.parse(file) / ET.fromstring(str); methods: getroot(), find(), findall(), iter(); element.tag, .attrib, .text.

```
import xml.etree.ElementTree as ET
xml = '''<college>
  <student id="1"><name>Anu</name><marks>85</marks></student>
  <student id="2"><name>Raj</name><marks>72</marks></student>
</college>'''
root = ET.fromstring(xml)
for st in root.findall("student"):
    print(st.attrib["id"], st.find("name").text, st.find("marks").text
new = ET.SubElement(root, "student", id="3")    # add element
ET.SubElement(new, "name").text = "Sam"
print(ET.tostring(root).decode())
```

| JSON | XML |
|---|---|
| Key-value, lighter and faster | Tags, verbose and heavier |
| Supports arrays, easy to parse | No native arrays; attributes + comments supported |
| Used in web APIs (REST) | Documents, config, SOAP, RSS |

## 4. NumPy with Python

* NumPy (Numerical Python) = library for fast N-DIMENSIONAL ARRAY (ndarray) processing. Arrays hold items of ONE data type (homogeneous), stored in contiguous memory - much faster and compact than lists; support vectorised operations, broadcasting, linear algebra, random numbers, statistics. Install: pip install numpy; import numpy as np.

## Creating arrays

```
import numpy as np
a = np.array([1, 2, 3, 4])              # 1-D from list
b = np.array([[1, 2, 3], [4, 5, 6]])    # 2-D
z = np.zeros((2, 3))                    # all 0.0
o = np.ones((2, 2), dtype=int)          # all 1
r = np.arange(0, 10, 2)                 # [0 2 4 6 8]
l = np.linspace(0, 1, 5)                # [0. .25 .5 .75 1.]
e = np.eye(3)                           # identity 3x3
f = np.full((2, 2), 7)
rn = np.random.rand(2, 2)               # random in [0,1)
```

## Attributes: ndim, shape, size, dtype

| Attribute | Meaning | For b above |
|---|---|---|
| ndim | Number of dimensions (axes) | 2 |
| shape | (rows, columns) tuple | (2, 3) |
| size | Total number of elements | 6 |
| dtype | Data type of elements | int64 |
| itemsize | Bytes per element | 8 |

```
print(b.ndim, b.shape, b.size, b.dtype)    # 2 (2, 3) 6 int64
print(b.reshape(3, 2))                     # change shape
print(b.T)                                 # transpose
print(b.flatten())                         # 1-D copy
c = np.array([1, 2, 3], dtype=float)       # [1. 2. 3.]
```

## Indexing and slicing

```
a = np.array([10, 20, 30, 40, 50])
print(a[1], a[-1], a[1:4], a[::2])          # basic slicing
m = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print(m[1, 2])              # 6        element row 1, col 2
print(m[0])                 # [1 2 3]  first row
print(m[:, 1])              # [2 5 8]  second column
print(m[0:2, 1:3])          # [[2 3] [5 6]]  sub matrix
print(a[[0, 2, 4]])         # integer (fancy) indexing -> [10 30 50]
print(a[a > 25])            # boolean indexing -> [30 40 50]
a[a > 40] = 0               # modify with condition
```

* Slices are VIEWS (share memory); use .copy() for an independent copy.

## Operations and aggregate functions

```
x = np.array([1, 2, 3]); y = np.array([4, 5, 6])
print(x + y, x * y, x ** 2, x > 1)       # element-wise
print(np.dot(x, y))                      # 32  (dot product)
a = np.array([[3, 1, 4], [1, 5, 9]])
print(a.min(), a.max(), a.sum())         # 1 9 23
print(a.sum(axis=0), a.sum(axis=1))      # column / row sums
print(a.mean(), np.median(a), a.std())   # statistics
print(np.cumsum([1, 2, 3, 4]))           # [1 3 6 10]  cumulative sum
print(a.argmax(), np.sort(a))            # index of max, sort
```

## Broadcasting, random numbers and linear algebra

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print(a + 10)                          # scalar broadcast to all items
print(a + np.array([100, 200, 300]))    # row vector added to each row
np.random.seed(1)
print(np.random.randint(1, 10, 5))  # 5 random integers
print(np.random.randn(3))           # normal distribution
A = np.array([[1, 2], [3, 4]]); B = np.array([[5, 6], [7, 8]])
print(A @ B)                           # matrix product [[19 22] [43 50]]
print(np.linalg.det(A), np.linalg.inv(A))
print(np.sqrt(A), np.exp(A), np.sin(A))     # universal functions
print(np.concatenate([A, B]), np.vstack([A, B]), np.hstack([A, B]))
```

| | List | NumPy array |
|---|---|---|
| Types | Mixed | Same type |
| Speed / memory | Slower, more | Faster, compact |
| [1,2] * 2 | [1,2,1,2] | [2 4] (element-wise) |
| Math on whole data | Needs loops | Vectorised, broadcasting |

## 5. Pandas

* Pandas = library for data manipulation and analysis, built on NumPy. Two main structures: SERIES (1-D labelled array) and DATAFRAME (2-D table with rows and labelled columns, like Excel / SQL table; columns can have different types). import pandas as pd; pip install pandas.

```
import pandas as pd
s = pd.Series([10, 20, 30], index=["a", "b", "c"])
print(s["b"], s.mean())                  # 20 20.0
data = {"Name": ["Anu", "Raj", "Sam", "Ria"],
        "Age": [20, 21, 19, 22],
        "Marks": [85, 72, 90, 66]}
df = pd.DataFrame(data)
print(df)
```

```
   Name  Age  Marks
0   Anu   20     85
1   Raj   21     72
2   Sam   19     90
3   Ria   22     66
```

## Reading / writing and exploring

```
df = pd.read_csv("data.csv")       # also read_excel, read_json
df.to_csv("out.csv", index=False)
print(df.head(3))        # first 3 rows (default 5);  df.tail() last r
print(df.shape, df.columns.tolist(), df.dtypes)
df.info()                # columns, non-null counts, types
print(df.describe())     # count mean std min quartiles max
print(df.isnull().sum()) # missing values per column
df = df.dropna()         # or df.fillna(0)
```

## Data selection operations

```
print(df["Name"])                    # one column (Series)
print(df[["Name", "Marks"]])         # several columns (DataFrame)
print(df.loc[1])                     # row by LABEL
print(df.loc[0:2, ["Name", "Age"]])   # label slice (end included)
print(df.iloc[1:3, 0:2])             # by POSITION (end excluded)
print(df.iloc[0, 2])                 # 85
print(df[df["Marks"] > 70])          # Boolean filter
print(df[(df.Age > 19) & (df.Marks > 70)])   # multiple conditions
df["Result"] = df["Marks"] >= 70  # new column
df = df.sort_values("Marks", ascending=False)
print(df.groupby("Result")["Marks"].mean())
```

| loc | iloc |
|---|---|
| Label based | Integer position based |
| Slice includes the END label | Slice excludes the end position |
| df.loc[row_label, col_label] | df.iloc[row_no, col_no] |

## Cleaning, grouping, merging

```
df = pd.DataFrame({"dept": ["CSE", "ECE", "CSE", "ECE", "CSE"],
                   "sal": [50, 40, None, 45, 60]})
print(df["sal"].isnull().sum())           # 1 missing
df["sal"] = df["sal"].fillna(df["sal"].mean())    # fill missing
print(df.groupby("dept")["sal"].agg(["mean", "max"]))
print(df["dept"].value_counts())          # CSE 3, ECE 2
df["bonus"] = df["sal"].apply(lambda x: x * 0.1)
df = df.rename(columns={"sal": "salary"}).drop(columns=["bonus"])
other = pd.DataFrame({"dept": ["CSE", "ECE"], "hod": ["Rao", "Devi"]})
print(pd.merge(df, other, on="dept"))     # join like SQL
print(df.drop_duplicates(), df.T.shape)
```

| Pandas function | Use |
|---|---|
| head(n), tail(n) | First / last n rows |
| info(), describe() | Structure and statistics |
| loc, iloc | Select by label / position |
| sort_values(), groupby() | Sort / aggregate |
| dropna(), fillna() | Handle missing data |
| merge(), concat() | Combine DataFrames |
| apply(), map() | Apply a function |
| value_counts(), unique() | Frequency / distinct values |
| corr() | Correlation between numeric columns |

## Plotting with matplotlib

* matplotlib.pyplot draws 2-D graphs: plot() line, scatter() points, bar(), hist(), pie(); add title(), xlabel(), ylabel(), legend(), grid(); show() displays. A SCATTER plot shows how one attribute changes with respect to another (relationship / correlation).

```
import matplotlib.pyplot as plt
plt.scatter(df["Age"], df["Marks"], color="red")
plt.title("Age vs Marks"); plt.xlabel("Age"); plt.ylabel("Marks")
plt.show()
plt.plot(df["Age"], df["Marks"], marker="o")   # line plot
plt.grid(True); plt.show()
df.plot(kind="bar", x="Name", y="Marks")       # pandas plotting
plt.show()
```

```
x = [1, 2, 3, 4]; y = [10, 20, 25, 30]
plt.figure(figsize=(6, 4))
plt.subplot(1, 2, 1); plt.bar(["A", "B", "C", "D"], y, color="g")
plt.subplot(1, 2, 2); plt.pie(y, labels=["A", "B", "C", "D"], autopct=
plt.savefig("chart.png"); plt.show()
plt.hist([1, 2, 2, 3, 3, 3, 4, 4, 5], bins=5)     # histogram
```

| Plot | Function | Used for |
|---|---|---|
| Line | plt.plot() | Trend over time |
| Scatter | plt.scatter() | Relation between two variables |
| Bar | plt.bar() | Compare categories |
| Histogram | plt.hist() | Distribution of data |
| Pie | plt.pie() | Percentage share |

## Important questions

* 1. Explain functional programming; map, filter, reduce with examples.
* 2. Explain JSON and XML handling in Python with programs.
* 3. Create NumPy arrays; explain ndim, shape, size, dtype, slicing and indexing.

* 4. What is a Pandas DataFrame? Explain loc and iloc, head(), data selection.
* 5. Write a program to draw a scatter plot using matplotlib.

# 6. Sample Experiments (24 - 30)

## Exp 24: Check whether a JSON string contains a complex object

* A JSON value is COMPLEX if it is an object (dict) or array (list) - i.e. not a simple string / number / boolean / null.

```
import json
def is_complex(js):
    obj = json.loads(js)
    return isinstance(obj, (dict, list))
print(is_complex('{"a": 1}'))        # True
print(is_complex('[1, 2, 3]'))       # True
print(is_complex('"hello"'))         # False
print(is_complex('25'))              # False
# alternative: check any value inside is dict / list
```

## Exp 25: NumPy array creation using array()

```
import numpy as np
a1 = np.array([1, 2, 3, 4, 5])                    # 1-D
a2 = np.array([[1, 2, 3], [4, 5, 6]])             # 2-D
a3 = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])  # 3-D
a4 = np.array([1, 2, 3], dtype=float)
print(a1); print(a2); print(a3); print(a4)
```

## Exp 26: ndim, shape, size, dtype

```
a = np.array([[1, 2, 3], [4, 5, 6]])
print("ndim :", a.ndim)          # 2
print("shape:", a.shape)         # (2, 3)
print("size :", a.size)          # 6
print("dtype:", a.dtype)         # int64
```

## Exp 27: Slicing, integer and Boolean indexing

```
a = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
print(a[0:2, 1:])                # basic slicing [[20 30] [50 60]]
print(a[[0, 1, 2], [0, 1, 2]])  # integer indexing: diagonal [10 50 90
print(a[a > 50])                 # Boolean indexing [60 70 80 90]
```

## Exp 28: Min, max, sum, cumulative sum

```
a = np.array([5, 3, 8, 1, 9, 2])
print("Min :", a.min())           # 1
print("Max :", a.max())           # 9
print("Sum :", a.sum())           # 28
print("Cumulative sum:", np.cumsum(a))   # [ 5  8 16 17 26 28]
```

## Exp 29: Dictionary of lists -> DataFrame, explore

```
import pandas as pd
d = {"Name":   list("ABCDEFGHIJ"),
     "Age":    [20,21,19,22,20,23,21,22,19,20],
     "Marks":  [85,72,90,66,78,88,59,95,70,81],
     "Height": [160,172,165,170,158,180,175,169,162,177],
     "Weight": [55,68,60,65,52,75,70,62,58,72]}
df = pd.DataFrame(d)
print(df.head())                          # a) first 5 rows
print(df["Marks"])                        # b) select column
print(df[["Name", "Marks"]])
print(df.loc[2:4])                        # rows by label
print(df.iloc[0:3, 1:3])                  # by position
print(df[df["Marks"] > 80])               # condition
```

## Exp 30: Scatter and plot of two columns

```
import matplotlib.pyplot as plt
plt.scatter(df["Height"], df["Weight"], color="blue")
plt.xlabel("Height (cm)"); plt.ylabel("Weight (kg)")
plt.title("Height vs Weight (scatter)")
plt.show()
df2 = df.sort_values("Height")
plt.plot(df2["Height"], df2["Weight"], marker="o")   # plot
plt.title("Height vs Weight (line)"); plt.show()
# observation: weight increases as height increases
# (positive correlation)
```

Revision: map / filter / reduce; json.loads / dumps; ElementTree; ndarray attributes ndim shape size dtype; loc (label) vs iloc (position); head(); plt.scatter / plot.